

Day2 · 10차시

메모리 & 지식 + RAG 입문

두뇌(추론)와 손발(도구)을 갖춘 에이전트에 — 이제 기억(memory)과 지식(knowledge)을 더한다.
7·9차시에서 만든 추천 에이전트가 기억하고, 내 데이터에서 찾아오는 에이전트로.

SECTION 00

오프닝 — 기억도, 내 데이터도 모른다

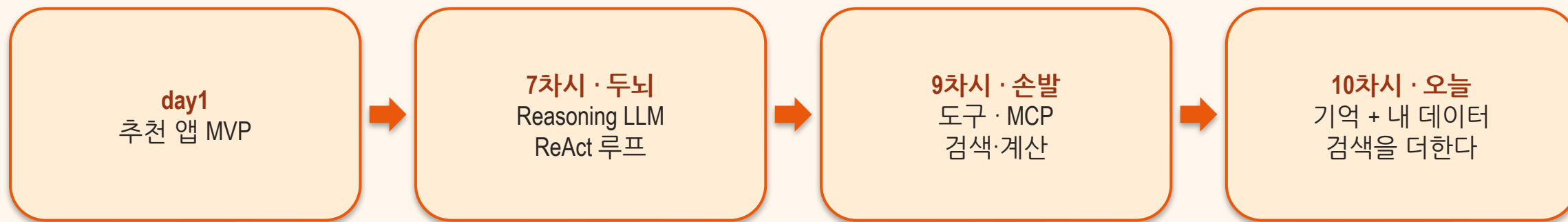
추론·도구는 갖췄다. 그런데 어제 한 말도, 우리 회사 문서도 모른다.

“방금 내가 뭐라고 했지?” (기억) · “그 정책, 우리 회사는 어떻게 돼 있지?” (지식) —
LLM은 둘 다 모른다.

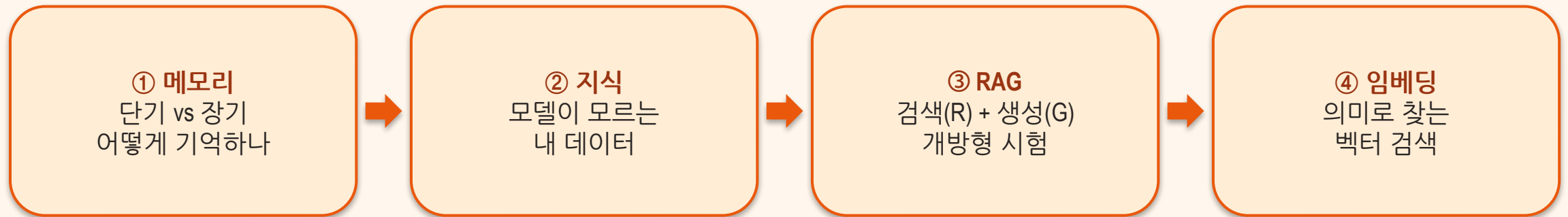
— LLM = 새 기억을 못 만드는 사람(영화 ‘메멘토’) — 학습 컷오프 이후·비공개 데이터는 못 본다

우리 에이전트, 어디까지 왔나

▶ 같은 맛집 추천 에이전트를 계속 발전 — 오늘은 기억(memory)과 내 데이터 검색(지식)



이 차시 동안 얻는 것



SECTION 01

메모리 — 에이전트는 어떻게 기억하나

메모리 vs 지식 — 무엇이 다른가

▶ 둘 다 결국 모델 컨텍스트를 풍부하게 — 보완적이지만 출처·성격이 다르다

메모리 (Memory)

- 에이전트가 겪은 일을 보관
- 과거 대화·도구 결과·상태
- 턴/세션을 잇는 연속성
- “내가 한 말을 기억”

VS

지식 (Knowledge)

- 대화 밖의 검증 가능한 사실
- 문서·정책·카탈로그·로그
- 보통 RAG로 구현
- “외부 사실을 가져옴”

LLM은 매 호출이 '백지'

▶ 기억은 엔지니어링의 결과 — 컨텍스트 윈도우 = 그 요청의 작업 기억(working memory)

LLM 그 자체 — stateless

- ROM은 있고 RAM은 없는 컴퓨터
- 호출이 끝나면 방금 대화를 잊음
- 매 호출이 백지에서 시작

VS

챗봇이 '기억하는 척' 하는 법

- 매번 전체 대화를 다시 첨부
- 기억의 착각 — 모델 능력이 아님
- '기억' = 컨텍스트를 다시 채워 넣는 일

사람처럼 — 기억의 3종류

▶ 오늘 다루는 건 일화적(메모리)과 의미적(지식·RAG) — 절차적은 시스템 프롬프트가 담당

일화적 (Episodic)

‘내가 무엇을 했나’
과거 대화·행동 로그
→ 대화 메모리

의미적 (Semantic)

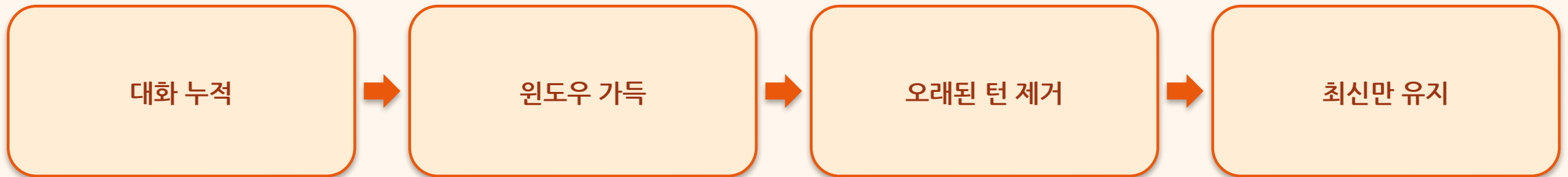
‘무엇이 사실인가’
문서·지식·정책
→ RAG의 토대

절차적 (Procedural)

‘어떻게 하나’
규칙·행동 방식
→ 시스템 프롬프트

롤링 컨텍스트 윈도우 (FIFO)

▶ 가장 단순한 메모리 = 컨텍스트 윈도우 자체 — 쉽지만, 오래되면 중요해도 사라진다



🔄 계속 쌓이며 가장 오래된 것부터 밀려남(선입선출)

컨텍스트 윈도우는 ‘아껴 쓸 자원’

▶ 길어질수록 중간 정보에 대한 집중력도 떨어진다 — 필요한 만큼만 넣는다

$\approx \frac{3}{4}$

토큰 1개
≈ 단어의 약 3/4 (≈4자)

750

1,000 토큰
≈ 영어 약 750 단어

1M

Gemini 2.5
입력 최대 100만 토큰

가득 차면 — 자르거나, 요약하거나

▶ 간격 반복(에빙하우스): 자주 쓰는 기억은 강해지고, 안 쓰면 흐려진다 (MemoryBank)

잘라내기 (Trimming)

- 오래된 것부터 삭제 (FIFO)
- 구현이 가장 쉽다
- 중요해도 오래되면 사라짐

VS

요약 (Summarization)

- 긴 대화를 주기적으로 압축
- 요지는 보존, 토큰은 절약
- 세부 사항은 손실

세션 안에서 vs 세션을 넘어

▶ 장기 메모리가 있어야 '복불 없이' 이어지는 자연스러운 경험이 된다

단기 메모리

- 이번 세션 한정
- 대화 누적·롤링 윈도우
- 구현 쉬움
- 세션 끝나면 사라짐

VS

장기 메모리

- 요약(summary): 긴 대화를 압축·핵심 보존
- 영속(persist): 세션을 넘어 저장·재개
- 사용자 선호·과거 맥락 기억
- '복불 없이' 이어지는 UX

SECTION 02

모델은 내 데이터를 모른다

LLM의 약점은 ‘맹목성(blindness)’

▶ 모르는 걸 물으면 그럴듯하게 지어냄 = 환각(hallucination)

아는 것

- 공개·과거 데이터(학습 범위)
- 일반 상식·널리 쓰인 코드
- 유창한 문장 생성

VS

모르는 것

- 사내 문서·비공개 지식
- 학습 컷오프 이후 최신 정보
- 우리 서비스의 데이터·정책

학습을 더 키우면 되지 않나 — 아니오

▶ 해법은 모델을 다시 가르치는 게 아니라, 답할 때 필요한 사실을 끌어오는 것

모델 재학습

- 세상 모든 정보 학습 불가
- 끝나는 순간 이미 늙음
- 내 데이터 주입 = 비싸고 위험
- 바뀔 때마다 다시?

VS

필요할 때 가져오기 (RAG)

- 질문 시점에 관련 사실만 검색
- 최신·비공개 데이터 즉시 반영
- 재학습 없이 저렴·빠름
- → 해법은 끌어오는 것

SECTION 03

RAG — 검색하고, 생성하라

RAG = Retrieval + Generation

▶ R(검색): 내 데이터에서 관련 사실을 먼저 찾는다 → G(생성): 그 사실만 근거로 답을 만든다



폐쇄형 시험 vs 개방형 시험

▶ 여러분 회사 데이터 질문이라면 — 당연히 오픈북 학생이 정확하다

순수 LLM (closed-book)

- 외운 것만으로 답
- 가중치 속 지식(parametric)
- 모르면 지어냄
- 최신·비공개엔 약함

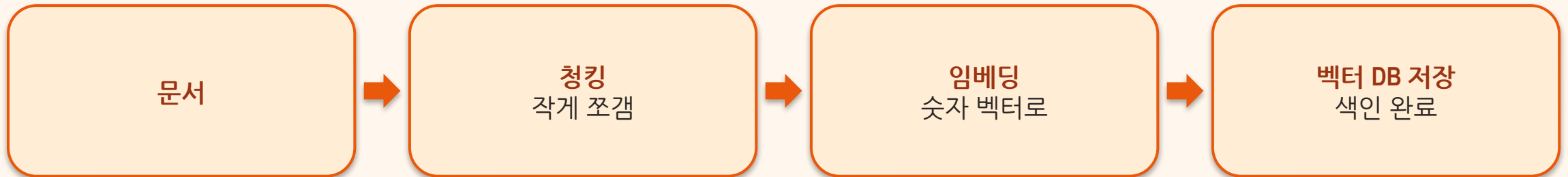
VS

RAG (open-book)

- 교재를 펴 보고 답
- 검색으로 실시간 공급된 지식
- 근거 위에서 답
- 내 데이터에 정확

인제스션 — 미리 한 번

▶ 긴 문서를 작은 청크로 나눠 임베딩(숫자 벡터)으로 바꾸고 벡터 DB에 색인



쿼리 — 질문마다

▶ 질문도 같은 모델로 임베딩 → 가까운 청크 top-k → 질문과 합쳐 생성 — ‘수집은 한 번, 질의는 매번’



🔄 질문이 올 때마다 매번 — 같은 임베딩 공간에서 비교

RAG가 바꾸는 것

▶ 재학습 없이 — 환각↓ 신뢰↑

RAG 없음

- 내 데이터 모름 → 지어냄
- 출처 없음 → 검증 불가
- 최신 반영 안 됨

VS

RAG 있음

- 도메인·회사 정보를 답에 반영
- 출처/인용 함께 제시
- 재학습 없이, 환각↓ 신뢰↑

오늘의 미니 RAG vs 프로덕션 RAG

▶ 데모는 10분, 프로덕션은 그때부터 — 오른쪽 기법들은 11차시에서

오늘 — 미니 RAG

- 메모 몇 줄 + numpy 코사인
- 청킹·임베딩·top-k·생성
- 원리는 프로덕션과 동일

VS

11차시 — 프로덕션 RAG

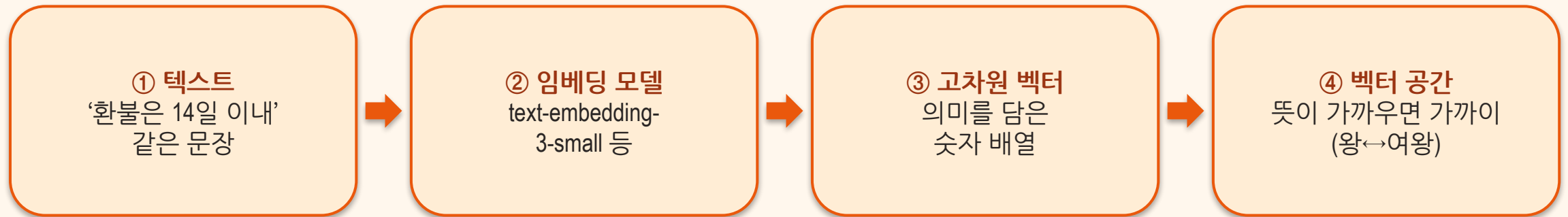
- 파싱·청킹 전략·벡터 DB
- 하이브리드 검색 + 리랭킹
- 평가(정확도·충실성)·환각 감지

SECTION 04

임베딩 & 벡터 검색 — 의미로 찾는다

의미를 좌표로 — 텍스트 → 벡터

▶ 키워드 검색은 '신입'↔'갓 들어온 사람'처럼 단어가 안 겹치면 못 찾는다 → 그래서 의미 기반 검색



벡터 검색 = 코사인 유사도 top-k

▶ 질문을 같은 모델로 임베딩 → 저장된 벡터들과 코사인 유사도 계산 → 가장 가까운 top-k



임베딩 + 코사인 (실습과 동일)

▶ 이 몇 줄에 의미 검색이 다 들어 있다 — 실습에서 직접 돌려본다

```
def embed(texts):  
    r = client.embeddings.create(model="text-embedding-3-small", input=texts)  
    return np.array([d.embedding for d in r.data])  
  
def cosine(a, b):  
    return float(a @ b / (np.linalg.norm(a) * np.linalg.norm(b)))  
# 질문 임베딩 → 모든 문서와 cosine → 상위 k
```

메모리 vs RAG — 무엇을 검색하나

▶ 기법(임베딩 검색)은 같아도 대상이 다르다: 내 경험 vs 외부 지식

메모리(경험)

- 검색 대상 = 내 과거 대화·상호작용
- ‘내가 전에 뭐라 했지’
- 개인화·연속성
- 겪은 일을 다시 꺼냄

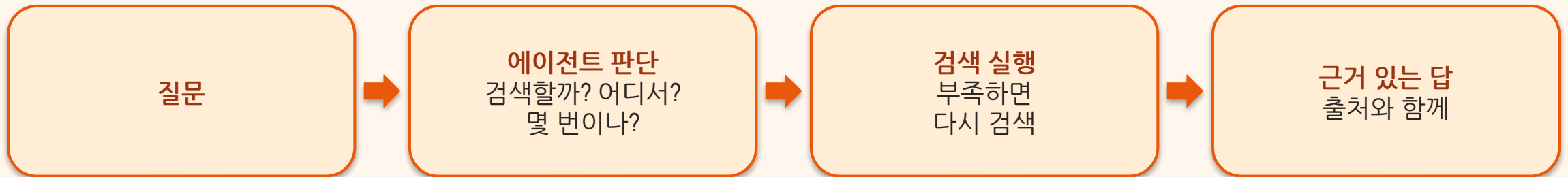
VS

RAG(지식)

- 검색 대상 = 외부 문서·사실
- ‘우리 정책이 뭐지’
- 근거·출처
- 대화 밖 사실을 가져옴

Agentic RAG — 검색도 에이전트가 결정한다

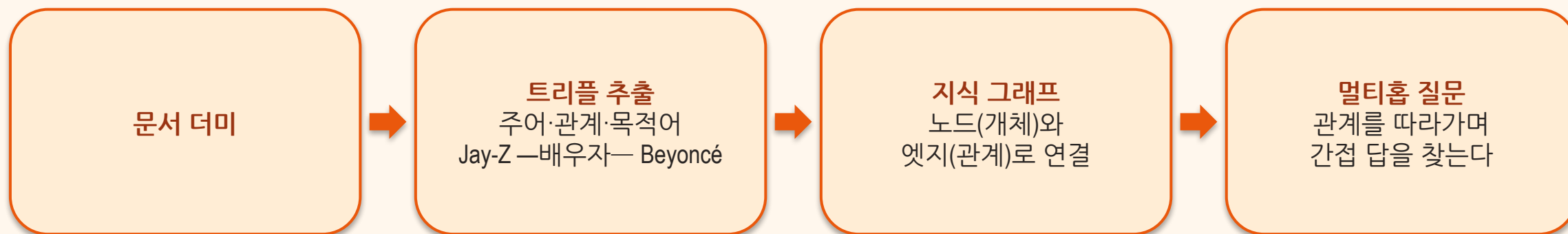
▶ RAG를 ‘한 번 돌리는 전처리’가 아니라 에이전트의 도구로



🔄 결과가 부족하면 검색을 반복 — 7차시 ReAct 루프에 검색 도구를 쫓은 것

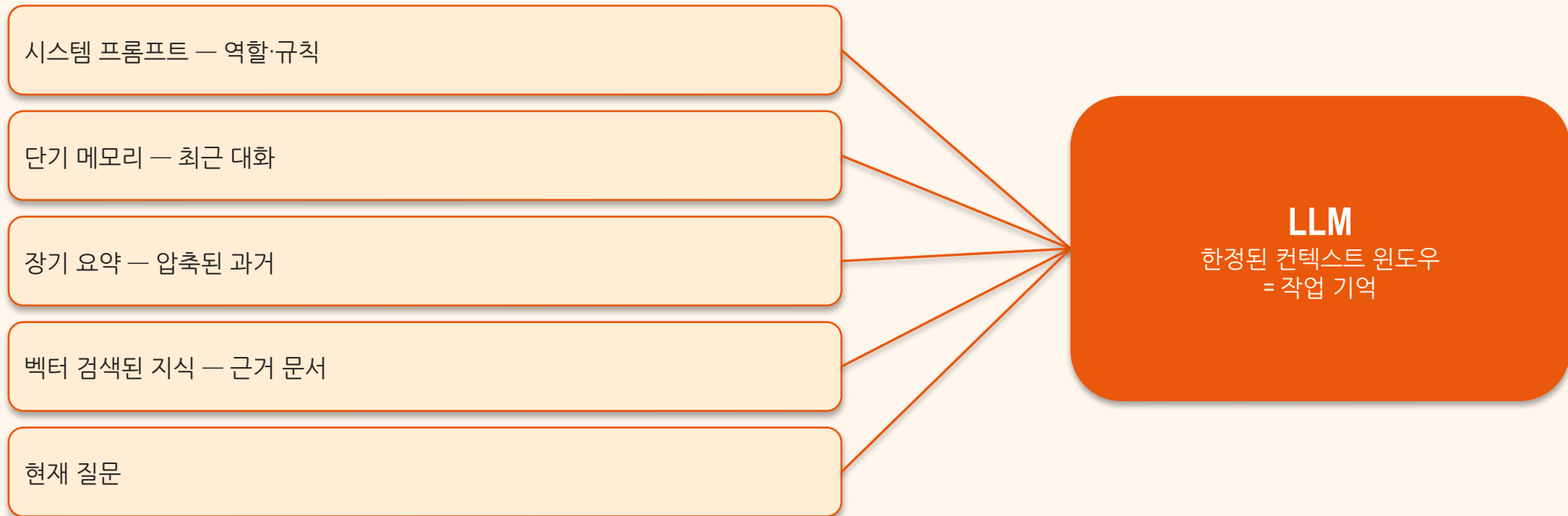
지식 그래프 — 관계로 잇는 지식 (GraphRAG)

▶ ‘Jay-Z의 배우자의 자녀는?’ — 여러 문서를 건너뛰는 질문(멀티홉)에 강하다



컨텍스트에 알맞게 채워 넣기

▶ 메모리·지식은 결국 컨텍스트 엔지니어링의 재료 — 한정된 윈도우에 알맞게 골라 채운다



SECTION 05

정리

오늘 3줄

- 메모리 = 겪은 일을 기억(단기 윈도우·요약·영속) · 지식 = **RAG(검색+생성)**
- 임베딩으로 텍스트를 의미 벡터로, 코사인으로 가까운 것을 찾는다(**의미 검색**)
- 둘 다 결국 **컨텍스트에 알맞게 채워 넣기** — 출력이 아니라 근거 있는 결과

모델을 다시 가르치지 말고, 답할 때 필요한 것을 가져오라.

— 기억(memory)은 겪은 일을, 지식(RAG)은 외부 사실을 — 컨텍스트에 채운다